VITE & GOURMAND

Traiteur Événementiel

DOCUMENTATION TECHNIQUE

Julie — Développeur Web et Web Mobile

Épreuve de Certification Finale (ECF)

Mars 2026

SOMMAIRE

1. Présentation technique du projet

Vite & Gourmand est une application web full-stack permettant la gestion complète d'un service de traiteur événementiel. La plateforme couvre l'ensemble du parcours client, de la consultation du catalogue à la livraison, en passant par la commande en ligne et la demande de devis personnalisés.

L'application est structurée selon une architecture client-serveur classique, avec un frontend React communiquant via API REST avec un backend Node.js/Express. Les données relationnelles sont stockées dans PostgreSQL, tandis que MongoDB assure le stockage des données analytiques.

Le déploiement est entièrement conteneurisé via Docker Compose, garantissant la reproductibilité de l'environnement sur n'importe quelle machine.

2. Stack technologique

Couche	Technologie	Rôle
Frontend	React 18 + Vite 5	Interface utilisateur SPA
Backend	Node.js + Express 4	API REST, logique métier
BDD relationnelle	PostgreSQL 16	Données principales
BDD analytique	MongoDB 7	Statistiques et agrégations
Authentification	JWT + bcrypt	Tokens d'accès et hachage
Email	Nodemailer + Mailhog	Envoi d'emails transactionnels
Tests	Jest + Supertest	Tests unitaires et d'intégration
Déploiement	Docker Compose	Conteneurisation multi-services
Sécurité	Helmet + CORS + Rate Limit	Protection HTTP
Logging	Pino	Journalisation structurée
Visualisation	Chart.js + react-chartjs-2	Graphiques analytiques admin
SEO	react-helmet-async	Balises meta par page

3. Architecture de l'application

L'application suit une architecture trois tiers classique : le frontend React (client) communique avec le backend Express (serveur) via des requêtes HTTP REST. Le backend interagit avec les bases de données PostgreSQL et MongoDB pour le stockage.

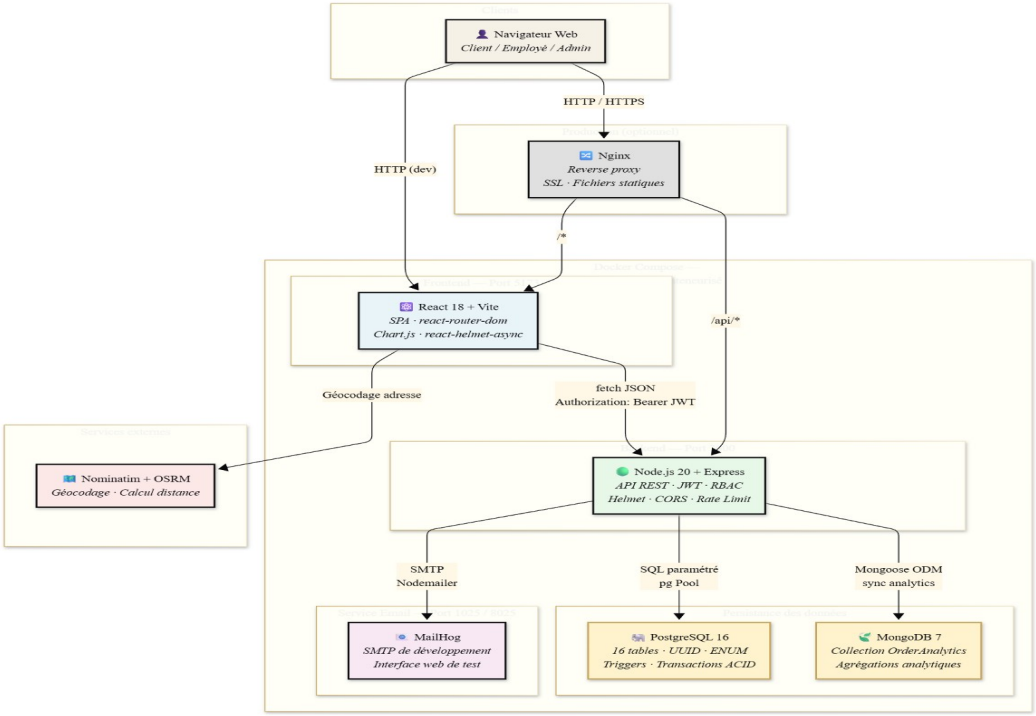

Figure 1 — Architecture générale de l'application

Le frontend est servi par Vite en développement (port 5173) et communique avec l'API backend (port 3000). Le backend gère l'authentification, la logique métier et l'accès aux données. PostgreSQL stocke les données relationnelles (utilisateurs, menus, commandes, devis) tandis que MongoDB stocke les données analytiques (agrégations de commandes, tendances).

4. Architecture backend OOP

Le backend adopte une architecture orientée objet organisée en quatre couches distinctes, chacune ayant une responsabilité bien définie :

Couche Controller

Les controllers reçoivent les requêtes HTTP, extraient les paramètres et délèguent le traitement au service correspondant. Ils ne contiennent aucune logique métier.

Controller	Méthodes principales
AuthController	register, login, refresh, forgotPassword, resetPassword, getMe
UserController	updateProfile, exportData, deleteAccount, listUsers, createEmployee, changePassword, toggleStatus

MenuController	list, getFilters, getById, create, update, delete
OrderController	list, getById, create, update, cancel, updateStatus, confirmDeposit, markPaid
QuoteController	list, getById, create, update, send, accept, refuse, deposit, convert, delete, expireOverdue
ReviewController	listApproved, listPending, create, validate
ContactController	create
AnalyticsController	getOrdersByMenu, getRevenue, getTrends, getClientScores
SuggestionController	menus, budget

Couche Service

Les services implémentent la logique métier : calculs de prix, transitions de statut, envoi d'emails, validation des règles fonctionnelles. Ils orchestrent les appels aux repositories et gèrent les transactions.

- **PricingService :** calcul du prix total (prix menu × convives + livraison − remise)
- **OrderService :** création de commande avec transaction, gestion du stock, machine à états
- **QuoteService :** workflow de devis (brouillon → envoyé → accepté → acompte → converti en commande)
- **EmailService :** 12 templates d'emails transactionnels via Nodemailer
- **AuthService :** inscription, connexion, refresh token, réinitialisation de mot de passe
- **AnalyticsService :** synchronisation et agrégation des données analytiques MongoDB

Couche Repository

Les repositories encapsulent l'accès aux données. Chaque repository utilise des requêtes SQL paramétrées (prévention des injections SQL) et expose un helper _db(client) permettant de basculer entre le pool global et un client de transaction.

Le projet compte 13 repositories : UserRepository, MenuRepository, DishRepository, OrderRepository, QuoteRepository, QuoteOptionRepository, ReviewRepository, ContactRepository, HoursRepository, LegalRepository, AnalyticsRepository, ClientScoreRepository et SuggestionRepository.

Injection de dépendances

Le fichier index.js sert de composition root : il instancie tous les repositories, services et controllers en injectant les dépendances via les constructeurs. Ce pattern facilite les tests unitaires et garantit un couplage faible entre les couches.

5. Routes API

L'API REST expose 13 groupes de routes. Chaque route est protégée par des middlewares d'authentification et d'autorisation selon les besoins.

Authentification (/api/auth)

Méthode	Endpoint	Accès	Description
POST	/register	Public	Inscription
POST	/login	Public	Connexion
POST	/refresh	Public	Refresh token
POST	/forgot-password	Public	Demande de reset
POST	/reset-password	Public	Réinitialisation
GET	/me	Auth	Profil connecté

Utilisateurs (/api/users)

Méthode	Endpoint	Accès	Description
PUT	/profile	Auth	Modifier profil
PUT	/password	Auth	Changer mot de passe
GET	/export	Auth	Export RGPD
DELETE	/account	Auth	Supprimer compte

GET	/	Admin	Lister utilisateurs
GET	/clients	Employé+	Lister clients
POST	/employee	Admin	Créer employé
PUT	/:id/toggle-status	Admin	Activer/Désactiver

Menus (/api/menus)

Méthode	Endpoint	Accès	Description
GET	/	Public	Catalogue avec filtres
GET	/filters	Public	Filtres disponibles
GET	/:id	Public	Détail menu + avis
POST	/	Employé+	Créer menu
PUT	/:id	Employé+	Modifier menu
DELETE	/:id	Employé+	Supprimer (soft)

Commandes (/api/orders)

Méthode	Endpoint	Accès	Description
GET	/	Auth	Liste commandes
GET	/:id	Auth	Détail commande
POST	/	Auth	Créer commande
PUT	/:id	Auth	Modifier commande
POST	/:id/cancel	Auth	Annuler commande
PUT	/:id/status	Employé+	Changer statut
POST	/:id/confirm-deposit	Employé+	Confirmer acompte

POST	/:id/mark-paid	Employé+	Marquer payé

Devis (/api/quotes)

Méthode	Endpoint	Accès	Description
GET	/	Auth	Liste devis
GET	/:id	Auth	Détail devis
POST	/	Auth	Créer brouillon
PUT	/:id	Auth	Modifier devis
POST	/:id/send	Employé+	Envoyer au client
POST	/:id/accept	Auth	Accepter devis
POST	/:id/refuse	Auth	Refuser devis
POST	/:id/deposit	Employé+	Enregistrer acompte
POST	/:id/convert	Employé+	Convertir en commande
DELETE	/:id	Auth/Admin	Supprimer devis

Avis (/api/reviews)

Méthode	Endpoint	Accès	Description
GET	/	Public	Avis approuvés
GET	/pending	Employé+	Avis en attente
POST	/:orderId	Auth	Déposer un avis
PUT	/:id/validate	Employé+	Valider/rejeter

Autres routes

Méthode	Endpoint	Accès	Description
POST	/api/contact	Public	Formulaire de contact
GET	/api/hours	Public	Horaires d'ouverture
PUT	/api/hours/:day	Employé+	Modifier horaires

GET	/api/legal/:type	Public	Page légale
PUT	/api/legal/:type	Admin	Modifier page légale
GET	/api/analytics/*	Admin	Données analytiques
GET	/api/suggestions/*	Public	Suggestions menus/budget

6. Base de données

PostgreSQL — Données relationnelles

La base de données PostgreSQL 16 stocke l'ensemble des données métier. Le schéma est initialisé automatiquement au démarrage du conteneur Docker via des scripts SQL montés dans /docker-entrypoint-initdb.d/.

Table	Description
users	Utilisateurs (clients, employés, admin) avec rôles, statut, consentement RGPD
menus	Catalogue de menus avec thème, régime, prix minimum, stock
menu_images	Images associées aux menus (URL)
dishes	Plats individuels (entrée, plat, dessert)
menu_dishes	Association menus ↔ plats (N:N)
orders	Commandes avec calculs de prix, statut, paiement
order_status_history	Historique des transitions de statut
order_contact_logs	Journal des contacts lors d'annulations
quotes	Devis avec détails événement, montants, validité
quote_items	Lignes de devis (menu, option, prestation)
quote_status_history	Historique des transitions de devis
quote_options	Options configurables pour les devis
reviews	Avis clients avec modération
contact_requests	Messages du formulaire de contact

business_hours	Horaires d'ouverture par jour (0–6)
legal_pages	Pages légales (mentions, CGV, confidentialité)

MongoDB — Données analytiques

MongoDB 7 stocke les données analytiques dans la collection order_analytics. Chaque commande créée est synchronisée de manière asynchrone (best-effort) vers MongoDB. Les agrégations permettent de générer les graphiques du dashboard administrateur : commandes par menu, chiffre d'affaires dans le temps, tendances de popularité.

7. Modèle Conceptuel de Données

Le MCD représente les entités principales du système et leurs relations. Les clés primaires sont des UUID générés par le backend.

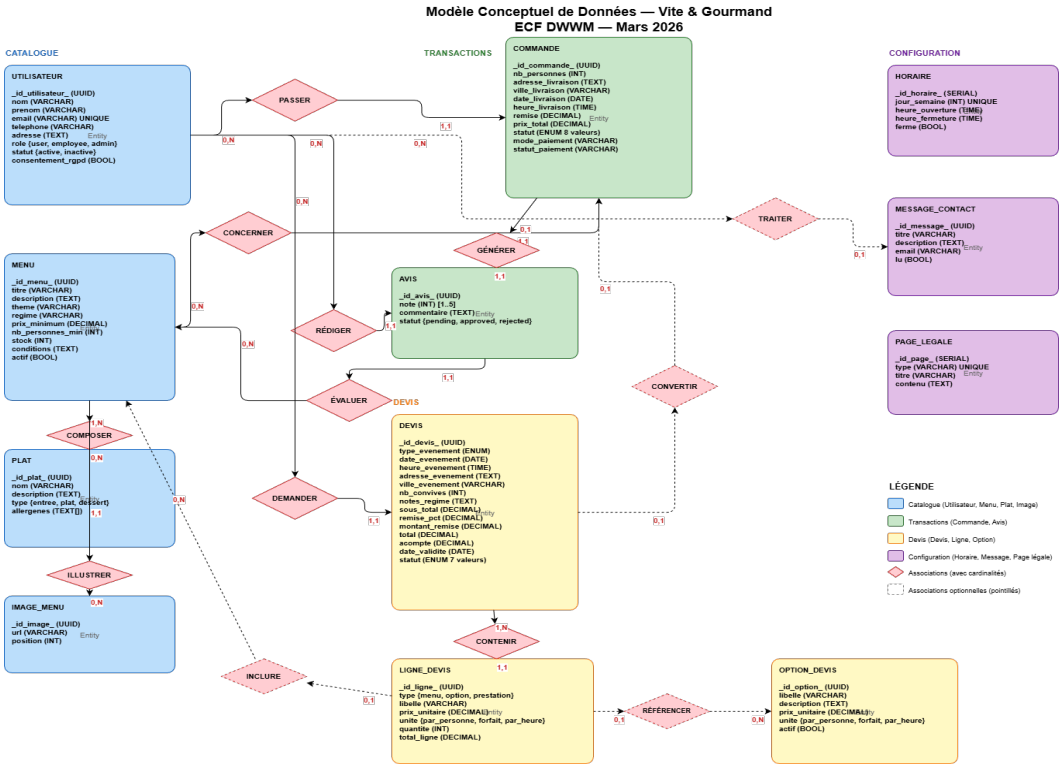

Figure 2 — Modèle Conceptuel de Données (MCD)

Le modèle illustre les relations principales : un utilisateur peut passer plusieurs commandes, chaque commande est liée à un menu, un menu contient plusieurs plats via une table de jointure, et les devis peuvent être convertis en commandes.

8. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les interactions entre les trois types d'acteurs (visiteur/client, employé, administrateur) et les fonctionnalités du système.

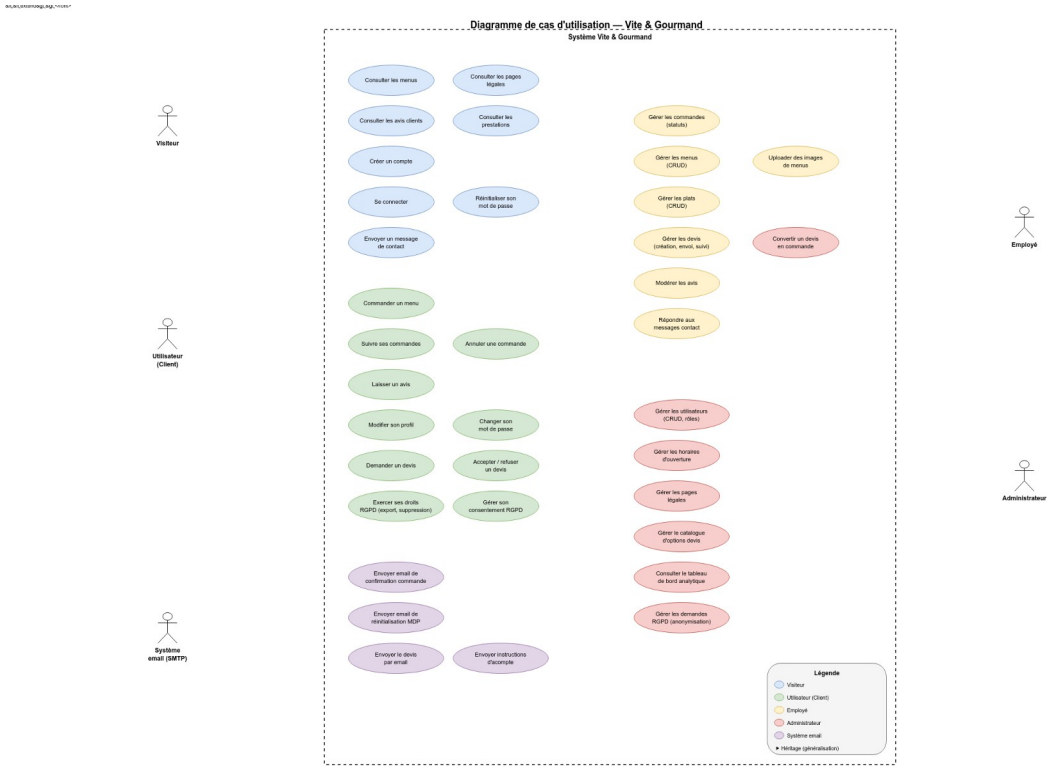

Figure 3 — Diagramme de cas d'utilisation

Le visiteur peut consulter les menus, créer un compte et soumettre un formulaire de contact. Le client authentifié peut en plus commander, demander un devis, suivre ses commandes et déposer des avis. L'employé gère les commandes et devis, tandis que l'administrateur supervise l'ensemble et accède aux analytiques.

9. Diagrammes de séquence

Commander un menu

Ce diagramme illustre le parcours complet de commande : sélection du menu, saisie des informations de livraison, calcul automatique du prix, création de la commande en transaction (décrément du stock, historique de statut) et envoi de l'email de confirmation.

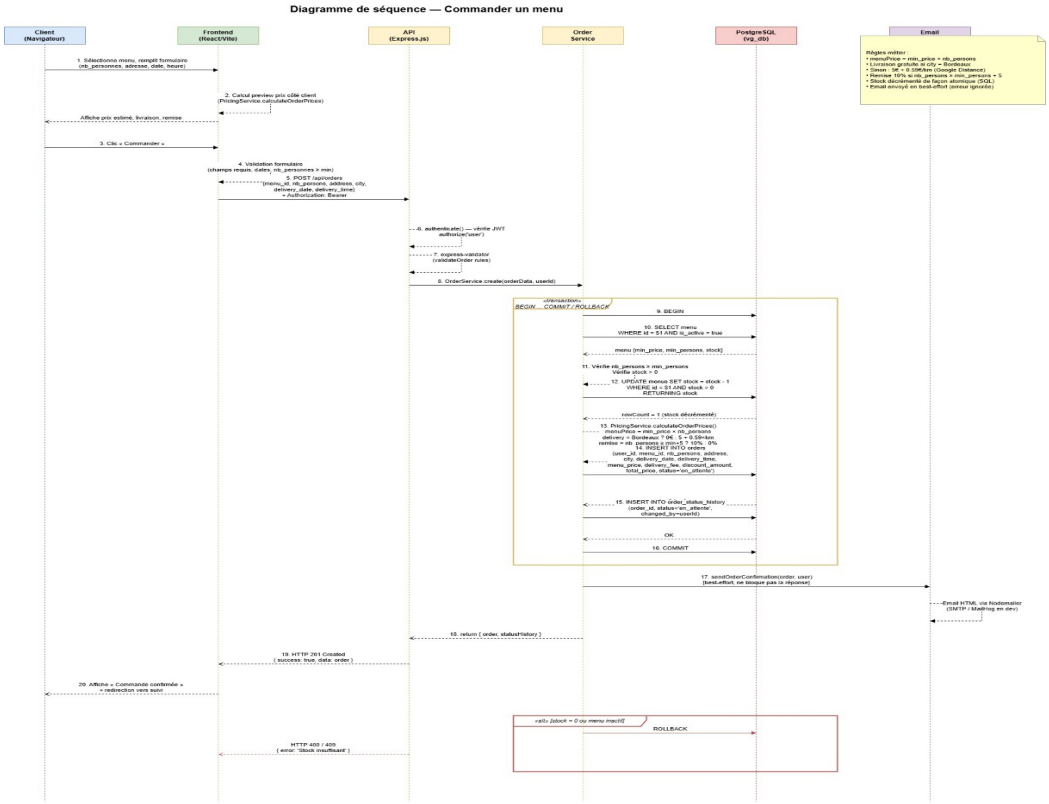

Figure 4 — Diagramme de séquence : commander un menu

Réinitialiser le mot de passe

Ce diagramme montre le flux de réinitialisation : demande par email (réponse non-révélatrice pour la vie privée), génération d'un token crypté (validité 1 heure), envoi du lien par email, et mise à jour du mot de passe haché avec bcrypt (salt 12).

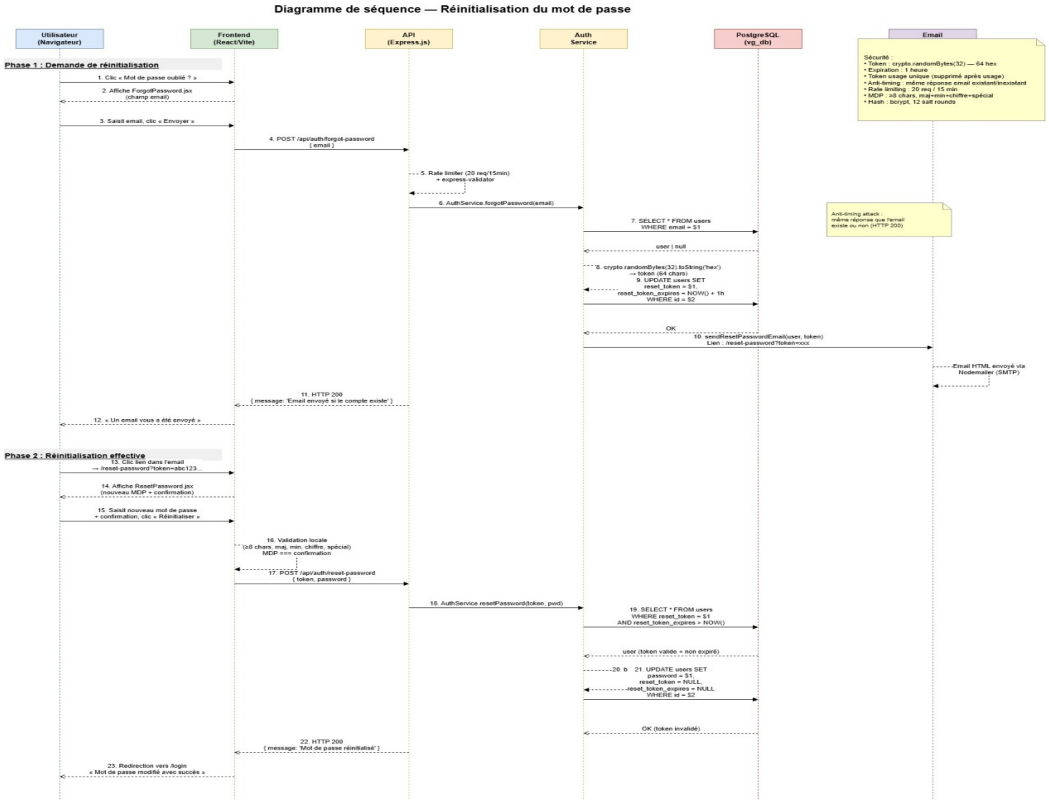

Figure 5 — Diagramme de séquence : réinitialisation du mot de passe

10. Sécurité

La sécurité de l'application repose sur plusieurs couches complémentaires :

Mesure	Implémentation
Hachage	bcrypt avec 12 rounds de sel (recommandation OWASP 2024)
Authentification	JWT HS256 — access token (24h) + refresh token (7 jours)
Autorisation	RBAC à 3 niveaux (user, employee, admin) + vérification de propriété
Headers HTTP	Helmet (CSP, X-Frame-Options, X-Content-Type-Options, HSTS)
CORS	Whitelist des origines autorisées via variable d'environnement
Rate Limiting	200 req/15min global, 20 req/15min sur les routes d'authentification
Injection SQL	Requêtes paramétrées exclusivement (librairie pg)
XSS	Échappement HTML dans les emails, Content-Type application/json
Validation	express-validator avec regex strictes (mot de passe ≥10 caractères)
RGPD	Export de données (art. 20), anonymisation de compte (art. 17), consentement
Reset password	Token crypto.randomBytes(32), expiration 1 heure, réponse non-révélatrice
Contact logs	Journalisation des annulations avec raison et méthode de contact

Règles de mot de passe

Le mot de passe doit respecter les critères suivants : minimum 10 caractères, au moins une majuscule, une minuscule, un chiffre et un caractère spécial. La validation est effectuée côté serveur via express-validator et côté client avec un indicateur de force en temps réel.

11. Déploiement Docker

L'application est déployée via Docker Compose avec 5 services interconnectés :

Service	Image	Port	Rôle
db_sql	PostgreSQL 16 Alpine	5432	Base relationnelle principale
db_mongo	MongoDB 7	27017	Base analytique
mailhog	Mailhog	1025/8025	Serveur SMTP de développement
api	Node.js (custom)	3000	Backend Express + API REST
frontend	Node.js (custom)	5173	Frontend React + Vite

Orchestration et dépendances

Docker Compose gère les dépendances entre services : l'API attend que PostgreSQL et MongoDB soient healthy avant de démarrer, et le frontend attend que l'API soit opérationnelle. Un health check sur /api/health vérifie la disponibilité du backend.

Volumes persistants

- pg_data — Persistance des données PostgreSQL
- mongo_data — Persistance des données MongoDB
- api_node_modules — Cache des packages backend
- frontend_node_modules — Cache des packages frontend

Initialisation de la base

Le schéma PostgreSQL est créé automatiquement via 5 scripts SQL montés dans le conteneur : 01-schema.sql (tables principales), 02-seed.sql (données de démonstration), 03-quotes.sql (tables de devis), 04-payments.sql (champs de paiement) et 05-quotes-deposit.sql (workflow d'acompte).

Commandes de déploiement

Le démarrage de l'application se fait en une seule commande :

```
docker compose up --build -d
```

L'application est ensuite accessible sur http://localhost:5173 (frontend) et http://localhost:3000/api (backend).

12. Tests

Le projet utilise Jest comme framework de test, complété par Supertest pour les tests d'intégration HTTP.

Fichier de test	Couverture
auth.validation.test.js	Validation des entrées d'authentification (email, mot de passe)
health.test.js	Vérification du endpoint /api/health
pricing.test.js	Calculs de prix (livraison, remise, total)
sanitize.test.js	Protection XSS et échappement HTML

Les tests sont exécutés via la commande npm test. Le mode watch (npm run test:watch) permet un feedback continu pendant le développement.